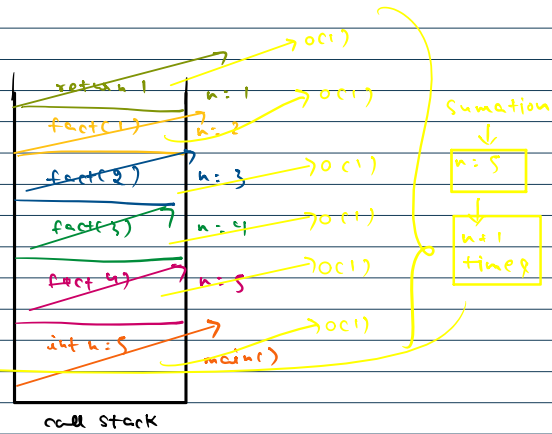


Recursive Time complexity

- 1. Single recursive call
- 2. Multiple recursive call.

1. Single recursive call

```
1 public class Factorial {
2     public static void main(String[] args) {
3         int n = 5;
4         int factorial = fact(n);
5         System.out.println(factorial);
6     }
7
8     public static int fact(int n) {
9         if (n == 1)
10            return 1;
11        return n * fact(n - 1);
12    }
13 }
14 }
```

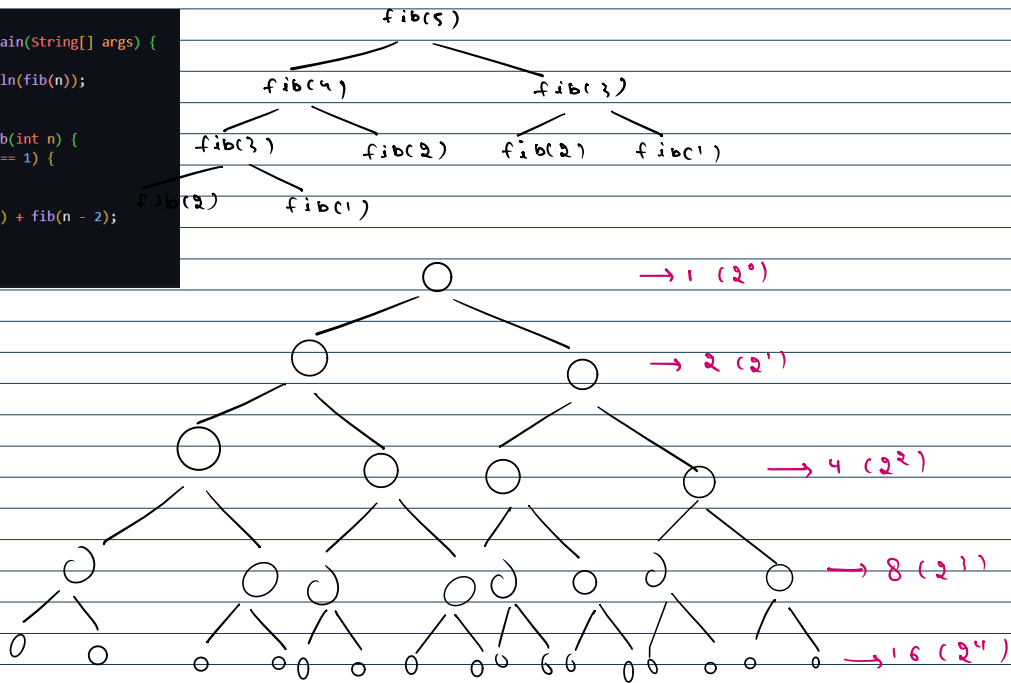


$1+1+1+...+n \text{ times}$
 $\Rightarrow n$
 $\downarrow$
 $O(n) \Rightarrow \text{Time complexity}$

2. Multiple recursive call

Total no. of recursive call.

```
1 public class Fibonacci {
2     public static void main(String[] args) {
3         int n = 5;
4         System.out.println(fib(n));
5     }
6
7     public static int fib(int n) {
8         if (n == 0 || n == 1) {
9             return n;
10        }
11        return fib(n - 1) + fib(n - 2);
12    }
13 }
14 }
```



Total no of recursive call

$$= (2^0 + 2^1 + 2^2 + 2^3 + 2^4) \times 1$$

$$= (2^0 + 2^1 + 2^2 + \dots + 2^n) \times 1$$

$$= 2^0 + 2^1 + 2^2 + \dots + 2^n$$

$$= \frac{1 \cdot (2^{n+1} - 1)}{2 - 1} = 2^{n+1} - 1$$

GP $\Rightarrow$ Geometric Progression
 $\downarrow$
 $r = 2^1 = 2$
Sum: $a \cdot \frac{r^n - 1}{r - 1}$

metric
gression

$$\frac{s^h - 1}{s - 1}$$

$$= \frac{1 \cdot (2^n - 1)}{(2 - 1)} = 2^n - 1$$

$\downarrow$
 2^n

$$\text{sum} = a \cdot C$$

$$(2 - 1)$$

$$r = \text{diff}$$

$$a = 1st +$$

Time: $O(2^n)$ Exponential

Recurrence Relation

Function of time which helps in finding time complexity.

fact(n) : fact(n-1) + constant time

$$T(n) = T(n-1) + 1 \rightarrow \text{Recurrence relation of factorial.}$$

$$\begin{matrix} T(1) = 1 \\ T(0) = 1 \end{matrix} \rightarrow \text{Base case}$$

$$T(n) = \begin{cases} T(n-1) + 1 & \forall n \neq 0, 1 \\ 1 & \forall n = 0 \end{cases}$$

Factorial

$$T(n) = T(n-1) + 1 \quad \text{Base case: input size is 1} \{T(1)\}$$

$$T(n) = T(n-1) + 1$$

$$T(n-1) = T(n-2) + 1$$

$$= [T(n-2) + 1] + 1$$

$$T(n-2) = T(n-3) + 1$$

$$= T(n-2) + 2$$

$$= [T(n-3) + 1] + 2$$

$$= T(n-3) + 3$$

⋮
→ after k expansion

$$T(n-k) = T(1) \Rightarrow n-k=1$$

$$k = n-1$$

$$T(n) = T(1) + k$$

$$T(n) = T(1) + n-1$$

$$T(n) = n-1$$

$$T(n) = n$$

$$T(n) = O(n)$$

Time complexity of factorial

Binary Search

$$T(n) = T\left(\frac{n}{2}\right) + 1 \quad \text{Base case: input size is 1} \{T(1)\}$$

$$T(n/2) = T(n/4) + 1$$

$$T(n) = T(n/2) + 1$$

$$= [T(n/4) + 1] + 1$$

$$T(n/4) = T(n/8) + 1$$

$$= T(n/4) + 2$$

$$= [T(n/8) + 1] + 2$$

$$= T(n/8) + 3$$

$$= T(n/2^3) + 3$$

⋮
→ after k expansion

$$T(n/2^k) = T(1) \Rightarrow \frac{n}{2^k} = 1 \Rightarrow n = 2^k$$

$$\log_2 n = \log_2 2^k$$

$$\log_2 n = k \cdot \log_2 2$$

$$k = \log_2 n$$

$$T(n) = T(1) + k$$

$$T(n) = \log_2 n + 1$$

$$T(n) = \log_2 n$$

$$T(n) = O(\log_2 n)$$

Time complexity of binary search

$$\frac{s^h - 1}{s - 1}$$

$$- 1)$$

$$\text{mult.})$$

$$e \times m$$

Merge Sort

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + n$$

merge 2 sorted arrays

Base case: input size is 1. $\{T(1)\}$

$$\begin{aligned} T(n) &= 2 \cdot T(n/2) + n \\ &= 2 \cdot [2 \cdot T(n/4) + n/2] + n \\ &= 2 \cdot 2 \cdot T(n/8) + \cancel{2} \cdot \left(\frac{n}{2}\right) + n \quad T(n/4) = 2 \cdot T(n/8) + n/4 \\ &= 4 \cdot T(n/8) + 2n \end{aligned}$$

$$= 4 [2 \cdot T(n/16) + n/4] + 2n$$

$$= 4 \cdot 2 \cdot T(n/16) + \cancel{4} \times \frac{n}{4} + 2n$$

$$= 8 \cdot T(n/16) + 3n$$

$$= 2^3 \cdot T(n/2^3) + 3n$$

→ after k expansion

$$T(n) = 2^k \cdot T(n/2^k) + k \cdot n$$

$$T(n) = 2^k \cdot T(1) + k \cdot n$$

$$T(n) = 2^{\log_2 n} \cdot T(1) + (\log_2 n) \cdot n$$

$$T(n) = n \cdot T(1) + n \cdot \log_2 n$$

$$T(n) = n + n \log_2 n$$

$$T(n) = O(n \log n)$$

Time complexity for merge sort

$$T\left(\frac{n}{2^k}\right) = T(1)$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k \Rightarrow k = \log_2 n$$

Fibonacci

$$T(n) = T(n-1) + T(n-2) + 1$$

Base case: input size is 1.

$$T(n) = T(n-1) + T(n-2) + 1$$

$$T(n) = [T(n-2) + T(n-3) + 1] + [T(n-3) + T(n-4) + 1] + 1$$

relation keeps growing → X

$$T(n-1) + T(n-2) + 1 \leq T(n-1) + T(n-1) + 1$$

↑
calculate time complexity

$$T(n) = 2 \cdot T(n-1) + 1$$

Base case: input size is 1 $\{T(1)\}$

$$\begin{aligned} T(n) &= 2 [2 \cdot T(n-2) + 1] + 1 \\ &= 4 \cdot T(n-2) + 3 \end{aligned} \quad T(n-1) = 2 \cdot T(n-2) + 1$$

$$T(n) = 4 \cdot [2 \cdot T(n-3) + 1] + 3 \quad T(n-2) = 2 \cdot T(n-3) + 1$$

$$= 4 \cdot 2 \cdot T(n-3) + 4 + 3$$

$$T(n) = 4 + 2 \cdot T(n-3) + 1 + 3$$

$$= 4 + 2 \cdot T(n-3) + 4 + 3$$

$$= 8 \cdot T(n-3) + 7$$

$$= 2^3 \cdot T(n-3) + (2^3 - 1)$$

→ after k
expansion

$$T(n-k) = T(1)$$

$$n-k = 1$$

$$k = n-1$$

$$T(n) = 2^k \cdot T(n-k) + 2^k - 1$$

$$= 2^{n-1} \cdot T(1) + 2^{n-1} - 1$$

$$= 2^n + 2^n$$

$$= 2^{n+1}$$

$$= 2^n$$

$$T(n) = O(2^n)$$

Time complexity
for fibonacci
series

→ pivot element is chosen randomly
instead of a fixed position.

Randomized Quick Sort

1 2 3 4 5

$$T(n) = T(n-1) + n$$

complexity
of partition
fn.

⇓

$$T(n) = O(n^2)$$

